IT项目管理

史建锋

2025年11月26日

传 统 估 算 方 法

1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

5.自下而上估算方法

内涵：利用任务分解图(WBS)，对各个具体工作包进行详细的成本估算，然后将结果累加起来得出项目总成本。

关键特点：

- ☑精度高（依赖细节拆解）
- ☑耗时较长（需全员参与）
- ☑适合中后期规划（当WBS已明确时）

操作步骤

① 拆解工作包：基于WBS（工作分解结构）分解到可估算的最小单元。

例如：开发一个登录功能 → 拆解为UI设计、前端开发、后端API、测试等任务。

① 估算单个成本：由负责该任务的成员或专家估算人力、时间、资源等成本。

② 逐级汇总：从最小任务向上汇总至模块、子系统，最终得到总成本。

③ 预留缓冲：针对风险较高的任务添加应急储备。

5.自下而上估算方法

- **IT项目案例：开发一个电商App的支付模块**

步骤1：拆解工作包

WBS层级	具体任务
1. 支付模块	
1.1 支付UI	设计支付页面、弹窗
1.2 支付逻辑	对接支付宝/微信支付API
1.3 安全验证	风控系统集成、Token加密
1.4 测试	单元测试、压力测试

步骤2：估算单个任务成本

任务	人力	工时（天）	成本（单价¥800/人天）
支付UI设计	1设计师	3	3×800 = ¥2,400
支付API对接	2后端工程师	5	2×5×800 = ¥8,000
安全验证开发	1安全工程师	4	4×800 = ¥3,200
测试	1测试工程师	3	3×800 = ¥2,400

步骤3：汇总总成本

- **支付模块总成本** = ¥2,400 + ¥8,000 + ¥3,200 + ¥2,400 = **¥16,000**
- **+10%应急储备** = ¥1,600 → **最终估算¥17,600**

- **自下而上估算的优缺点**

优点	缺点
精度高（误差通常<10%）	耗时耗力（需详细拆解和团队协作）
团队参与度高，估算更可信	依赖WBS的完整性（若漏任务则低估）
易于跟踪和控制单个任务成本	不适用于早期模糊需求阶段

传 统 估 算 方 法

1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

6．三点估算方法

- 基于任务成本的三种估算值来计算预期成本的方法。

① 最可能成本（C_M）：比较现实的估算成本。

② 最乐观成本（C_O）：最好情况所得到的估算成本。

③ 最悲观成本（C_P）： 最差情况所得到的估算成本。

- 三角分布（用于描述不确定性或风险）：$C_E=(C_O+C_M+C_P)/3$
- 贝塔分布（默认，常作为先验信息）：$C_E=(C_O+4C_M+C_P)/6$

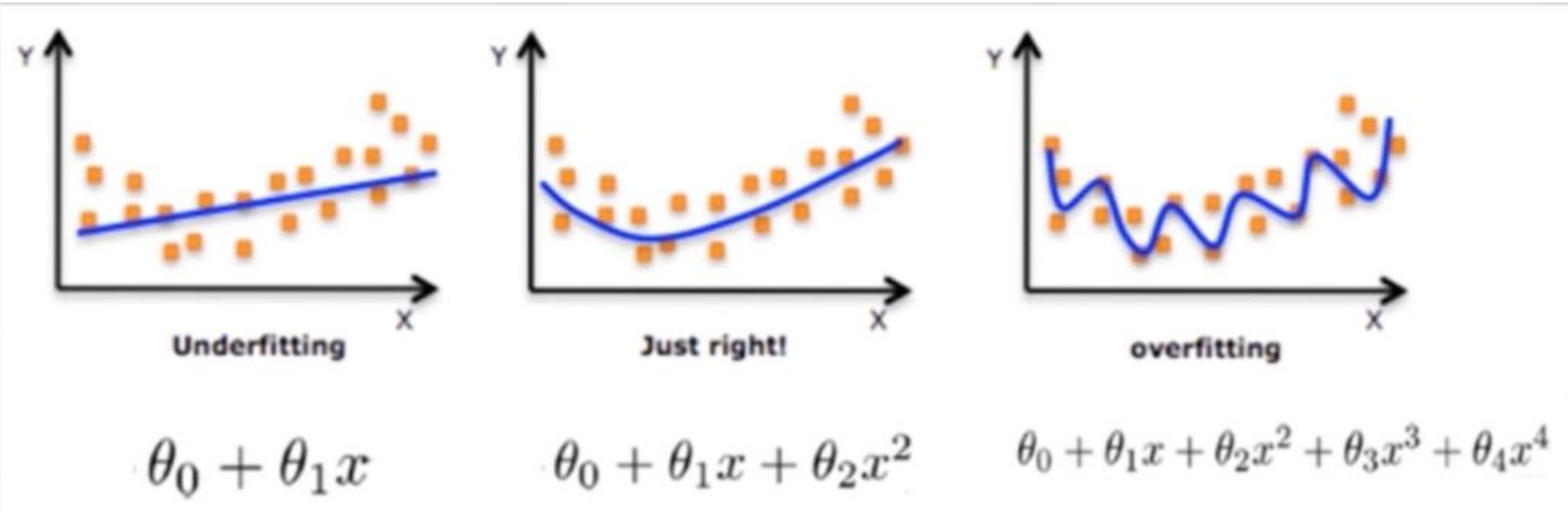
例如：$C_O=7$, $C_P=12$, $C_M=9$

三角分布: $C_E=9.33$，贝塔分布: $C_E=10.67$

传 统 估 算 方 法

1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法

- 通过已有项目数据，进行分析，得出参数估算模型；
- 通过参数模型估算(规模)成本的方法。



需要掌握的二个典型模型

- 小型到中型项目：Walston-Felix模型（IBM）
- 中大型复杂项目：COCOMO模型

Walston-Felix模型

IBM的Walston和Felix提出了如下的估算公式：

- $E = 5.2 \times (KLOC)^{0.91}$，KLOC是源代码千行数，E是工作量（以人月计）
- $S = 0.54 \times E^{0.6}$，S是人员需要量(以人计)
- $DOC = 49 \times (KLOC)^{1.01}$ ，DOC是文档数量(以页计)

Walston-Felix模型举例

采用java完成项目，估计有366功能点，则

$$L = 366 \times 48 = 17568\text{行} = 17.568\text{KLOC}$$

$$E = 5.2 \times (\text{KLOC})^{0.91} = 5.2 \times 17.568^{0.91} = 70\text{人月}$$

$$DOC = 49 \times (\text{KLOC})^{1.01} = 49 \times 17.568^{1.01} = 886\text{页}$$

功能点与代码行的转换

编程语言	SLOC/FP	编程语言	SLOC/FP
Assembly	320	Object languages	30
C	128	Smalltalk	22
Cobol	105	generation languages	20
Fortran	105	Powerbuilder	16
Pascal	90	Koodigenerators	15
Ada	70	SQL	12
C++	64	Spread sheat pogram	6
Ada 95	53	Graphical language	4
Visual Basic	32	**Java**	48
C#	36	VB	29
Delphi	18	代码生成器	15

COCOMO（ Constructive Cost model ）

- 结构化成本模型
- 是目前应用最广泛的参数型软件成本估计模型
- 由Barry Boehm团队开发，包含二种

① COCOMO-81

② COCOMO-II

COCOMO 81

模型级别:

- 基本COCOMO
- 中等COCOMO
- 高级COCOMO

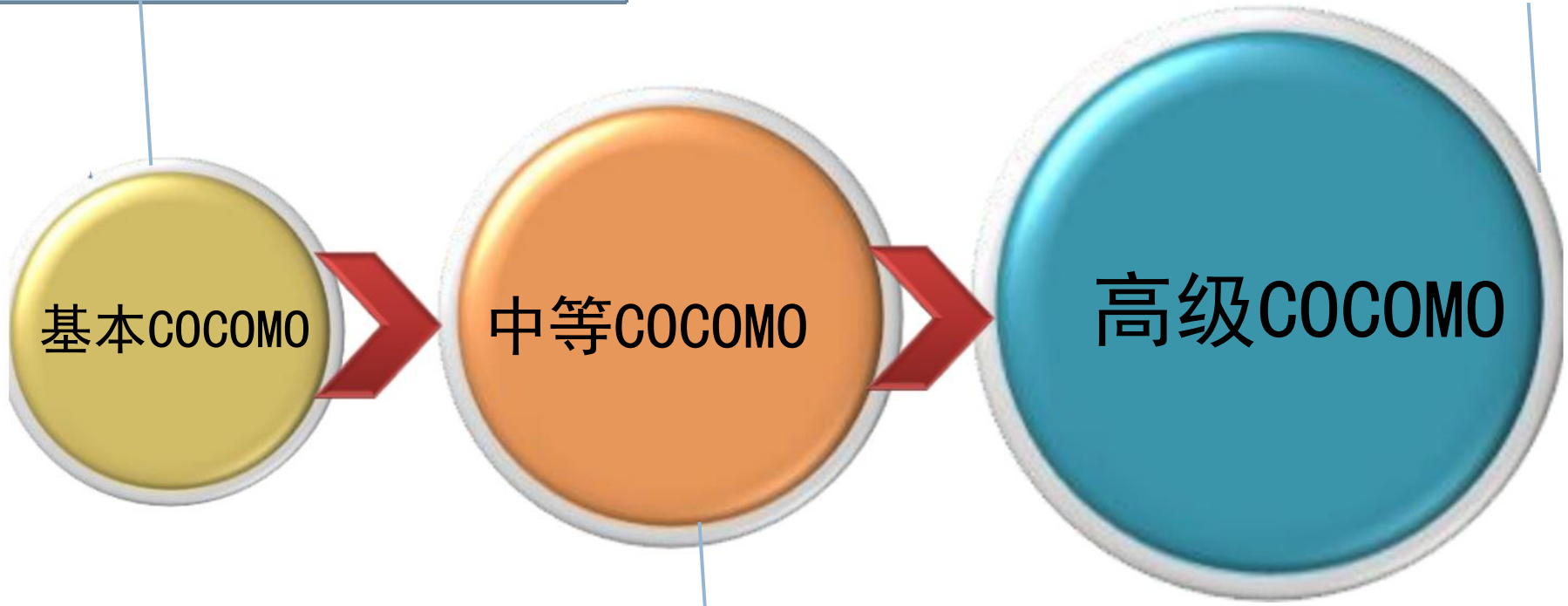
项目类型:

- 有机：Organic
- 嵌入式：Embedded
- 半嵌入：Semidetached

7．参数估算方法

静态单变量模型

在中等COCOMO模型基础上考虑各个步骤的影响



基本模型基础上考虑影响因素，调整模型

COCOMO-81项目类型

- 有机：Organic
  - 各类应用程序，例如数据处理、科学计算等
  - 受硬件的约束比较小，程序的规模不是很大
- 嵌入式：Embedded
  - 系统程序，例如实时处理、控制程序等
  - 紧密联系的硬件、软件和操作的限制条件下运行，软件规模任意
- 半嵌入式：Semidetached
  - 各类实用程序，介于上述两种软件之间，例如编译器（程序）
  - 规模和复杂度都属于中等或者更高

➢ 基本COCOMO-81

$$E = a \times (KLOC)^b$$

- E：工作量（人月）
- KLOC：是交付的代码行
- a，b：依赖于项目自然属性的系数

方式	a	b
有机	2.4	1.05
半有机	3.0	1.12
嵌入式	3.6	1.2

例

一个33.3KLOC的软件开发项目，属于中等规模、半有机型的项目，采用基本COCOMO，计算项目规模。

a=3.0， b=1.12。

E = 3.0 * L ^1.12 = 3.0 * 33.3 ^1.12 = 152 PM（人月）

➢ 中等COCOMO-81

$E = a \times (KLOC)^b \times$乘法因子

- E：工作量（人月）
- KLOC：是交付的代码行
- a， b：依赖于项目自然属性的系数
- 乘法因子：对公式的校正系数

方式	a	b
有机	2.4	1.05
半有机	3.0	1.12
嵌入式	3.6	1.2

乘法因子的成本驱动属性



乘法因子 = $F_1 \times F_2 \times \ldots\ldots \times F_{15}$

Cost drivers	Rating levels						Remark
	Very low	Low	Nominal	High	Very high	Extra high	
Product attributes							
RELY	0.75	0.88	1.00	1.15	1.40		Required software reliability
DATA		0.94	1.00	1.08	1.16		Database size
CPLX	0.70	0.85	1.00	1.15	1.30	1.65	Product complexity
Computer attributes							
TIME			1.00	1.11	1.30	1.66	Execution time constraints
STOR			1.00	1.06	1.21	1.56	Main storage constraints
VIRT		0.87	1.00	1.15	1.30		Virtual machine volatility
TURN		0.87	1.00	1.07	1.15		Computer turnaround time
Personnel attributes							
ACAP	1.46	1.19	1.00	0.86	0.71		Analyst capability
AEXP	1.29	1.13	1.00	0.91	0.82		Applications experience
PCAP	1.42	1.17	1.00	0.86	0.70		Programming capability
VEXP	1.21	1.10	1.00	0.90			Virtual machine experience
LEXP	1.14	1.07	1.00	0.95			Language experience
Process attributes							
MODP	1.24	1.10	1.00	0.91	0.82		Use of modern programming practices
TOOL	1.24	1.10	1.00	0.91	0.83		Use of software tools
SCED	1.23	1.08	1.00	1.04	1.10		Required development schedule

举例

一个33.3KLOC的软件开发项目，属于中等规模、半有机型的项目，采用中等COCOMO，计算项目规模。

a=3.0，b=1.12。

乘法因子＝0.70*0.85*1……*1.15=1.09

E=3.0×L^1.12×乘法因子=3.0×33.3^1.12×1.09=166PM

➢ 高等COCOMO-81

- 将项目分解为一系列的子系统或者子模型
- 更加精确地调整一个模型的属性，将软件工程过程中分析、设计等步骤的影响考虑进来，不同阶段驱动因子不同。

成本驱动因子	开发阶段	经验水平				
		很低	低	正常	高	很高
AEXP	需求设计阶段	1.4	1.2	1	0.87	0.75
	详细设计阶段	1.3	1.15	1	0.9	0.8
	编码和单元测试阶段	1.25	1.1	1	0.92	0.85
	集成测试阶段	1.25	1.1	1	0.92	0.85

COCOMO-II模型

COCOMO-II是Boehm在COCOMO-81模型的基础上，结合了软件工程技术的发展提出来的。COCOMO-II包含了应用组装模型、早期设计模型以及后体系结构模型，分别针对规划阶段、设计阶段以及开发阶段。



参数模型综述

- 根据项目数据进行回归分析，得出回归模型作为参数模型。
- 回归分析方法：线性回归，多项式回归，逻辑回归，神经网络方法等。
- 参数模型可以是线性也可以是非线性的。

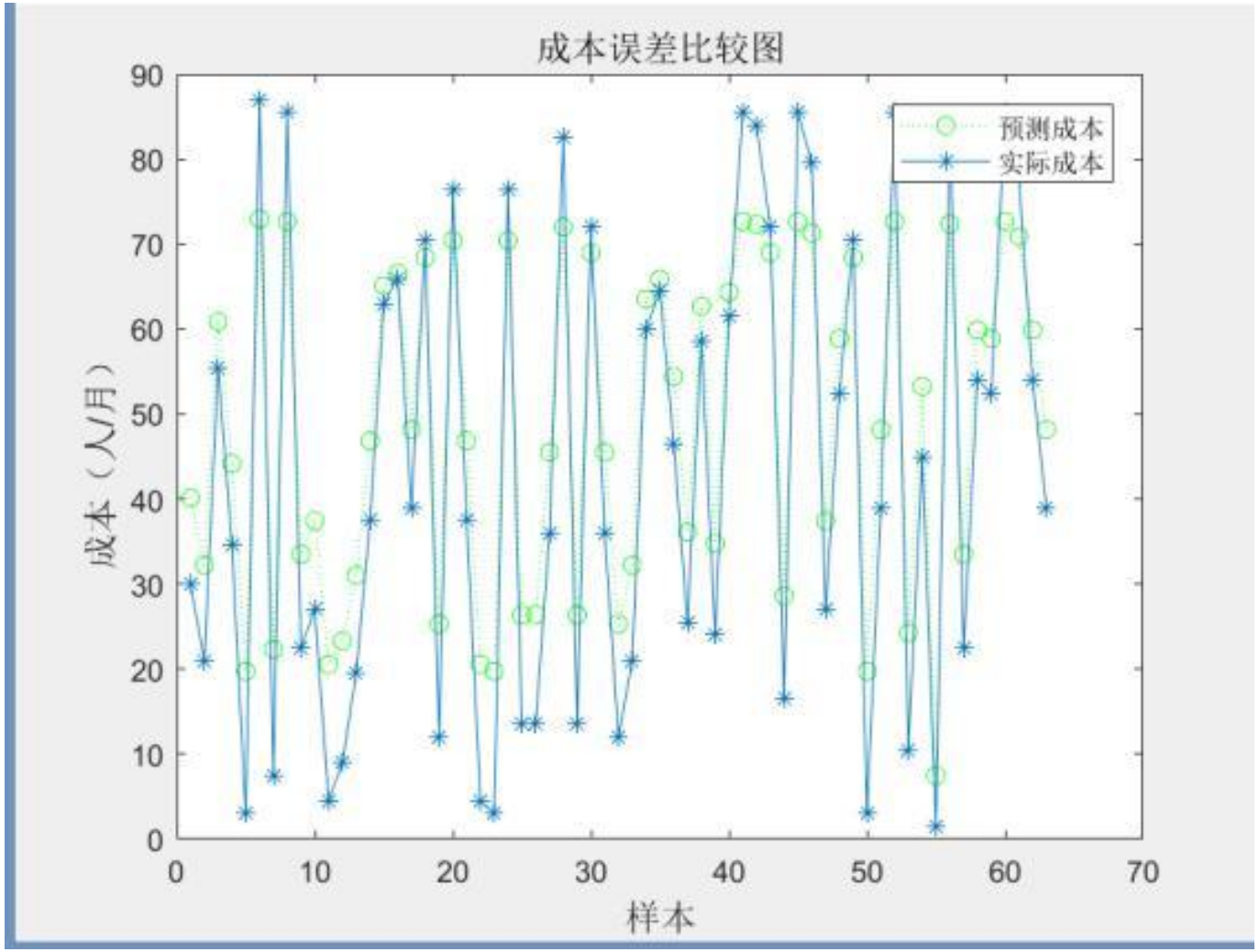
基于神经网络估算模型-项目数据

	A	B	C	D	E	F	G	H	I	J	K	L
1	功能点数	38	54	31	47	58	1	51	45	39	34	3
2	可靠性	很高	高	很低	很低	很低	低	低	很低	低	很低	高
3	数据规模	高	很高	很低	极高	极高	正常	低	很高	很低	低	很低
4	复杂性	高	低	高	正常	很高	很低	正常	低	很低	高	正常
5	文档	低	正常	极高	低	极高	高	极高	很高	很高	很低	高
6	复用	正常	很低	极高	很低	极高	低	低	低	很高	正常	极高
7	技术难度	很低	很高	低	极高	极高	高	正常	高	低	很高	正常
8	平台难度	低	很低	低	很低	极高	极高	低	极高	高	很高	很低
9	需求易变性	很低	很低	高	很高	很低	极高	很低	高	低	低	高
10	分析能力	低	极高	正常	正常	高	很高	正常	低	极高	正常	正常
11	应用经验	高	正常	正常	低	高	很高	极高	很低	极高	高	高
12	程序员水平	极高	高	极高	高	正常	高	很高	正常	正常	正常	很高
13	语言与工具经验	高	低	极高	高	低	很低	低	低	很低	正常	很高
14	团队凝聚力	极高	正常	很低	很低	低	高	高	高	正常	高	高
15	过程成熟度	正常	高	高	很高	低	极高	低	极高	低	高	正常
16	使用软件工具的水平	正常	极高	低	低	高	很低	正常	很低	低	低	正常
17	进度管理	很高	低	正常	正常	极高	极高	极高	极高	极高	低	正常
18												
19	时间（月）	17.6	25	14.4	24.6	29	0.4	24.6	26.1	18	15	1.7
20	成本（人/月）	52.8	75	43.2	73.8	87	1.2	73.8	78.3	54	45	5.1
21	缺陷数（个）	95	135	79	125	156	4	125	117	89	75	7

项目进行数字化和归一化

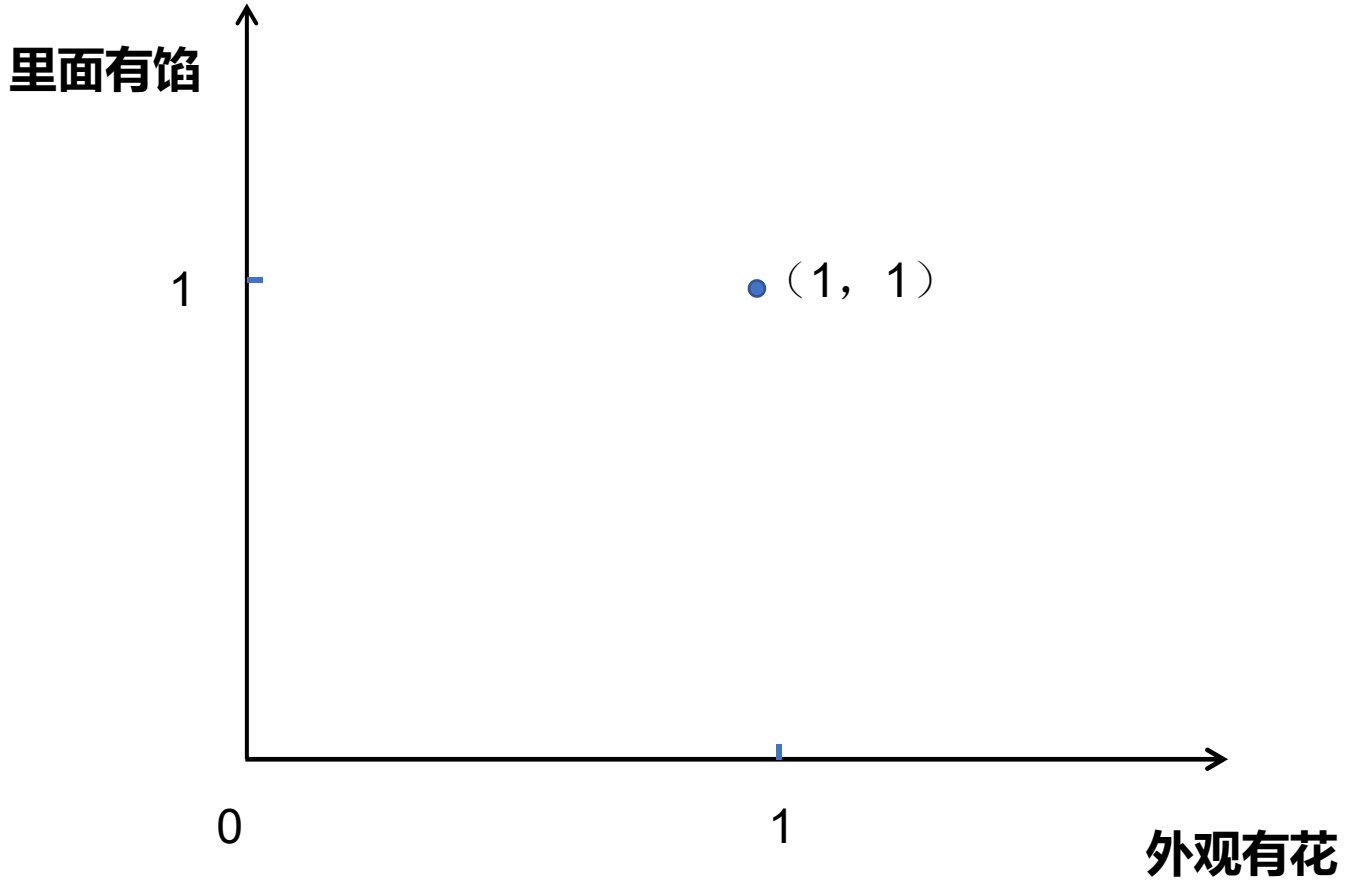
	A	B	C	D	E	F	G	H	I	J	K	L
1	功能点数	0.88	0.34	0.35	0.63	0.90	0.01	0.44	0.14	0.52	0.35	0.04
2	可靠性	0.65	0.13	0.51	0.43	0.43	0.40	0.20	0.13	0.07	0.05	0.63
3	数据规模	0.42	0.84	0.97	0.09	0.30	0.14	0.57	0.60	0.94	0.58	0.75
4	复杂性	0.90	0.50	0.23	0.83	0.42	0.32	0.75	0.33	0.26	0.06	0.56
5	文档	0.97	0.21	0.71	0.19	0.34	0.95	0.77	0.96	0.04	0.38	0.50
6	复用	0.45	0.74	0.21	0.29	0.32	0.93	0.43	0.90	0.49	0.25	0.42
7	技术难度	0.13	0.03	0.42	0.64	0.04	0.61	0.96	0.41	0.87	0.09	0.83
8	平台难度	0.34	0.90	0.46	0.10	0.92	0.72	0.06	0.69	0.22	0.80	0.43
9	需求易变性	0.62	0.53	0.17	0.69	0.85	0.92	0.10	0.62	0.16	0.84	0.47
10	分析能力	0.12	0.48	0.60	0.73	0.96	0.65	0.42	0.09	0.59	0.70	0.59
11	应用经验	0.49	0.25	0.07	0.16	0.00	0.28	0.40	0.40	0.65	0.57	0.58
12	程序员水平	0.82	0.06	0.71	0.37	0.74	0.66	0.06	0.57	0.04	0.15	0.47
13	语言与工具经验	0.15	0.57	0.32	0.09	0.27	0.62	0.94	0.08	0.64	0.52	0.01
14	团队凝聚力	0.07	0.89	0.15	0.19	0.46	0.63	0.75	0.05	0.75	0.81	0.11
15	过程成熟度	0.26	0.01	0.46	0.17	0.19	0.79	0.27	0.29	0.06	0.02	0.07
16	使用软件工具的水平	0.20	0.42	0.41	0.60	0.60	0.84	0.04	0.16	0.10	0.04	0.99
17	进度管理	0.29	0.10	0.47	0.41	0.94	0.17	0.24	0.75	0.84	0.36	0.85
18												
19	时间（月）	0.28	0.80	0.31	0.64	0.86	0.06	0.77	0.07	0.83	0.34	0.14
20	成本（人/月）	0.71	0.95	0.53	0.65	0.37	0.53	0.68	0.20	0.73	0.68	0.18
21	缺陷数（个）	0.95	0.96	0.71	0.85	0.87	0.88	0.96	0.32	0.08	0.29	0.54

模型应用

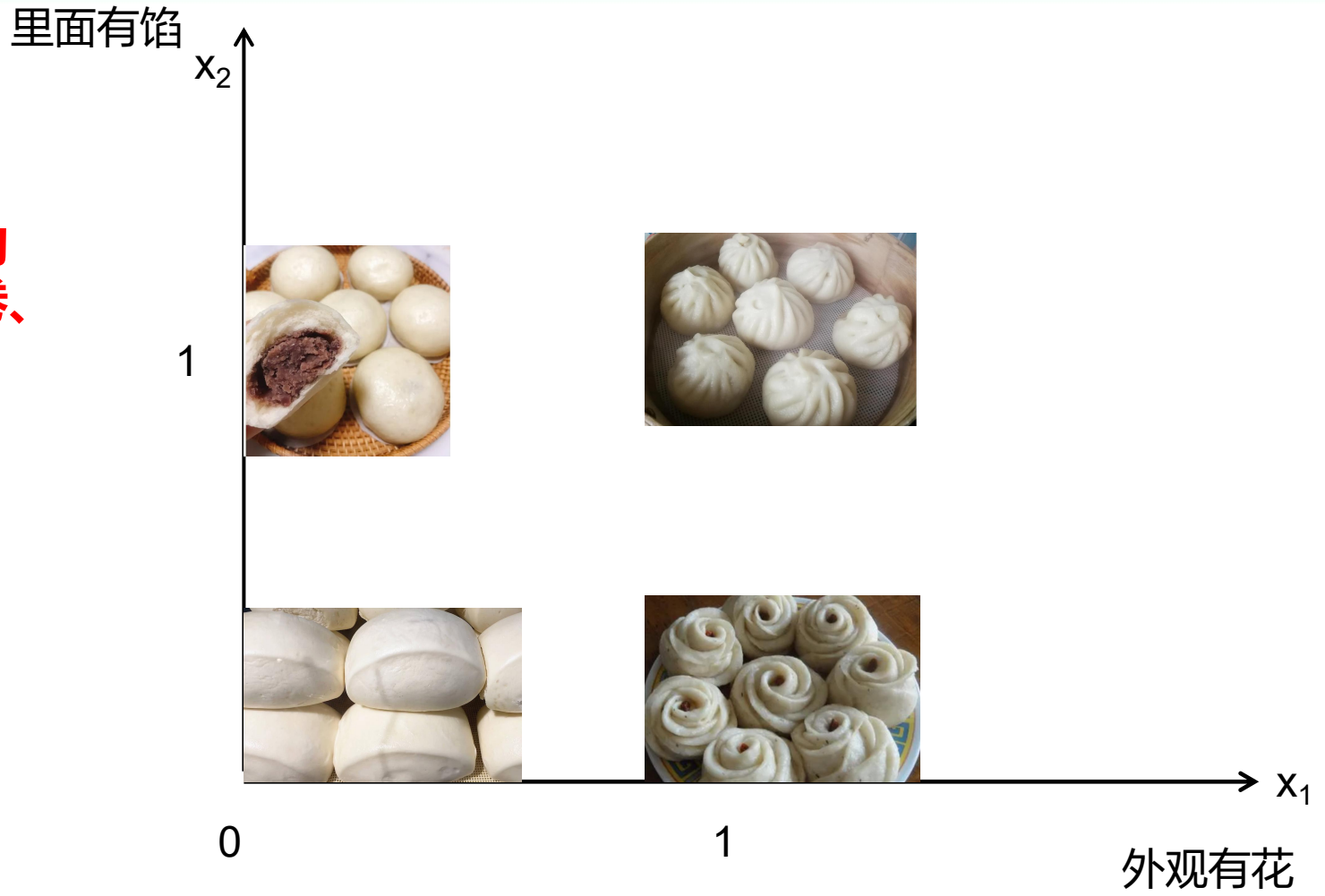


- 神经网络的原理

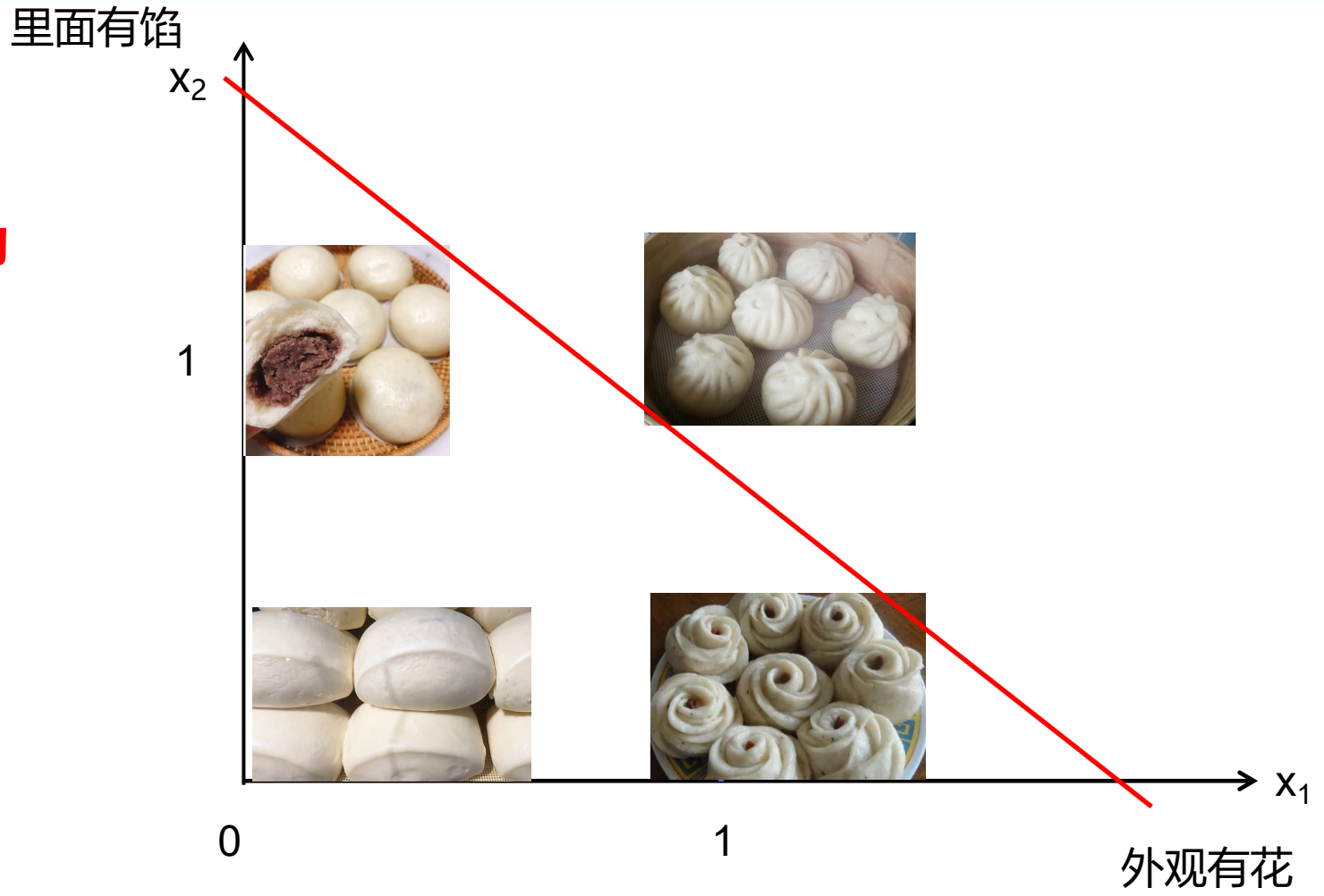
例：如何让机器区分常见的面食？（馒头、包子、花卷、豆沙包）



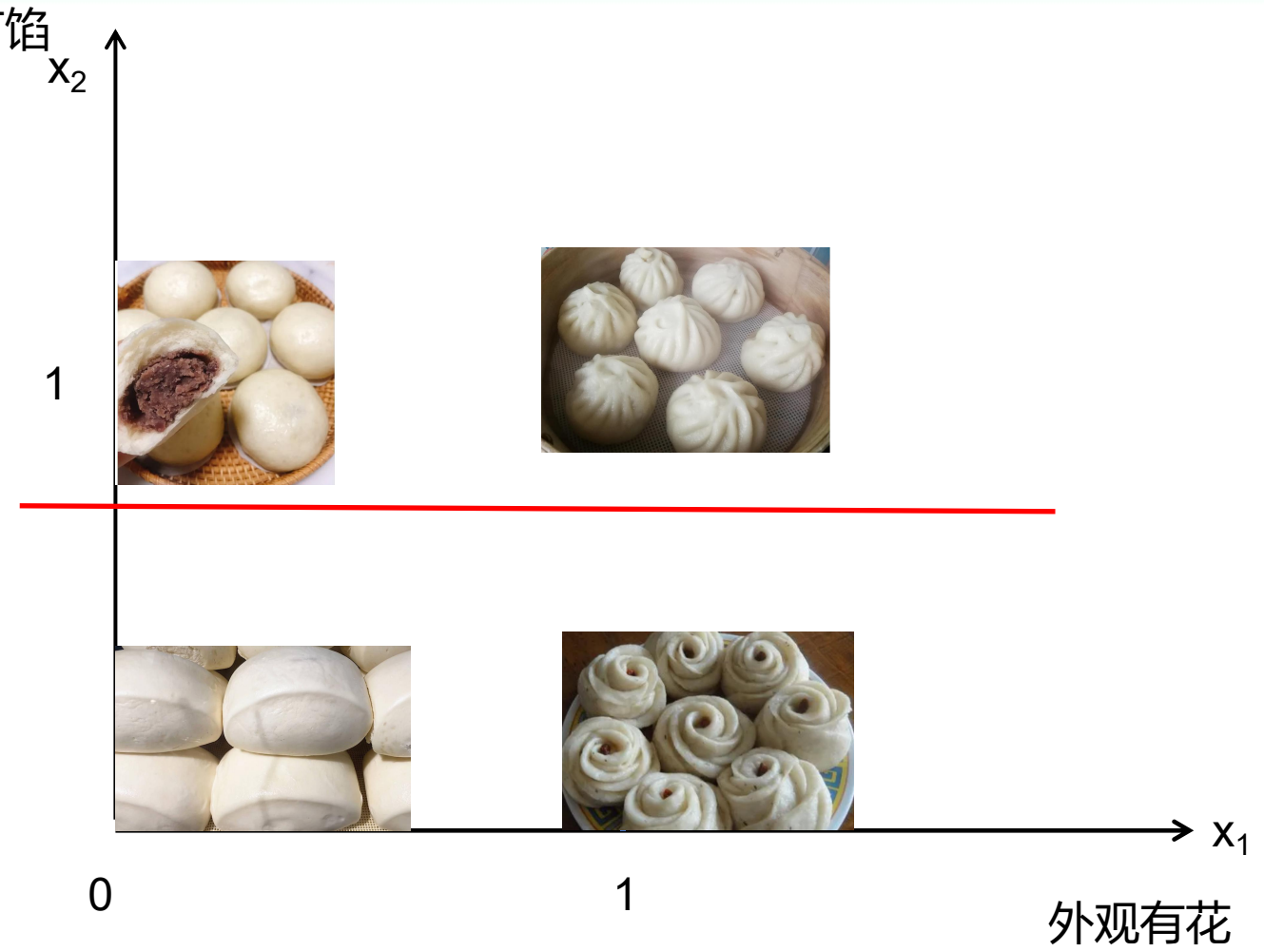
例：如何让机器区分常见的面食？（馒头、包子、花卷、豆沙包）



例：如何让机器区分常见的面食？（把包子单独区分）

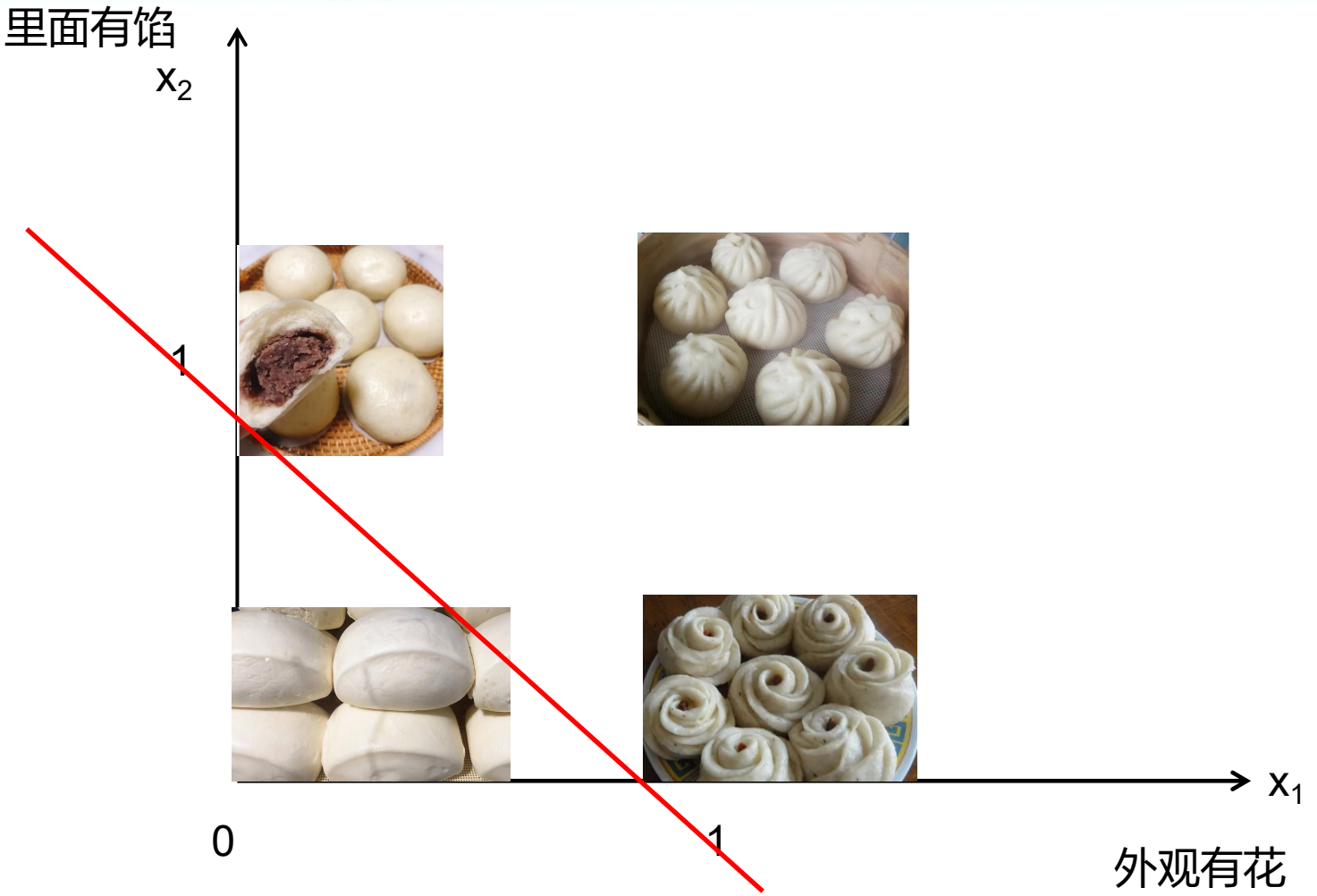


例：如何让机器区分常见的面食？（把带馅和不带馅的归类）

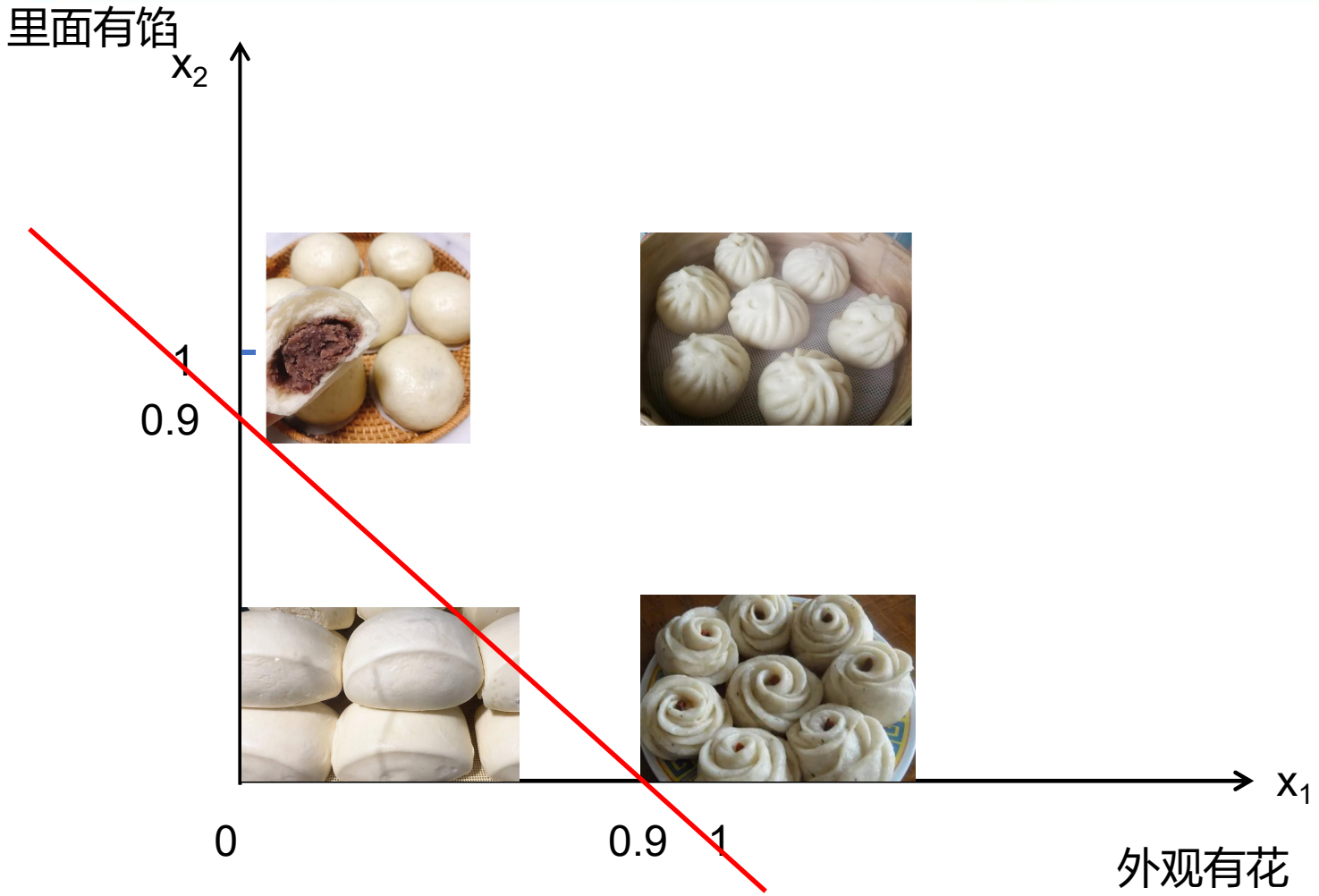


7．参数估算方法

例：如何让机器区分常见的面食？（把馒头区分出来）

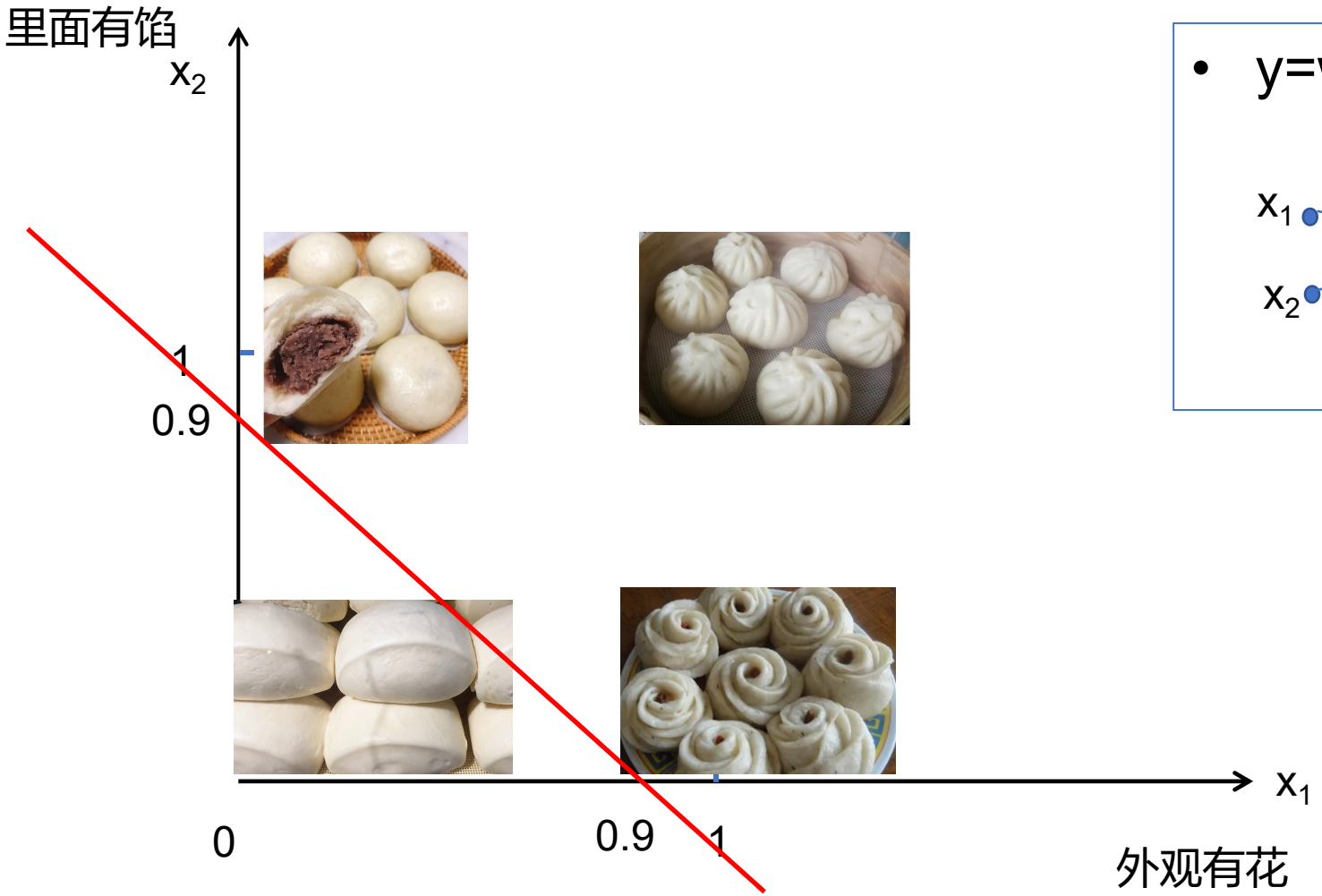


7．参数估算方法

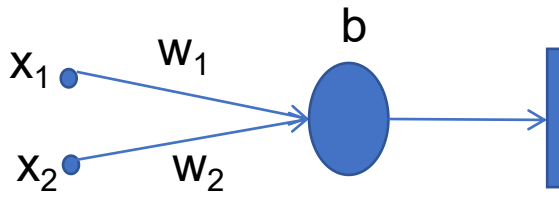


- $y=w_1 \cdot x_1+w_2 \cdot x_2+b$
- w_1:x_1的权重
- w_2:x_2的权重
- b:偏置量，调节作用
- 如果令w_1、w_2均为1，b=-0.9，则原式变为：$y=x_1+x_2-0.9$
- 由此可给出，当y<=0时为馒头，y>0时为其他

7．参数估算方法



- $y=w_1 \cdot x_1+w_2 \cdot x_2+b$



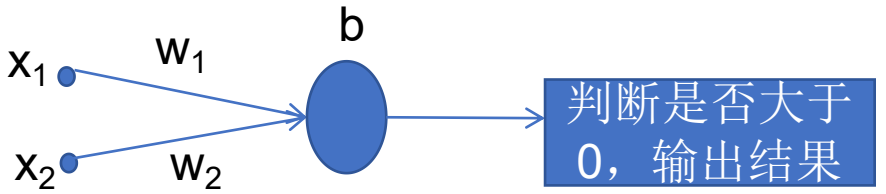
7．参数估算方法

里面有馅

如果我们所面临的问题是：馒头和包子是一类，豆沙包和花卷是一类，如何区分？



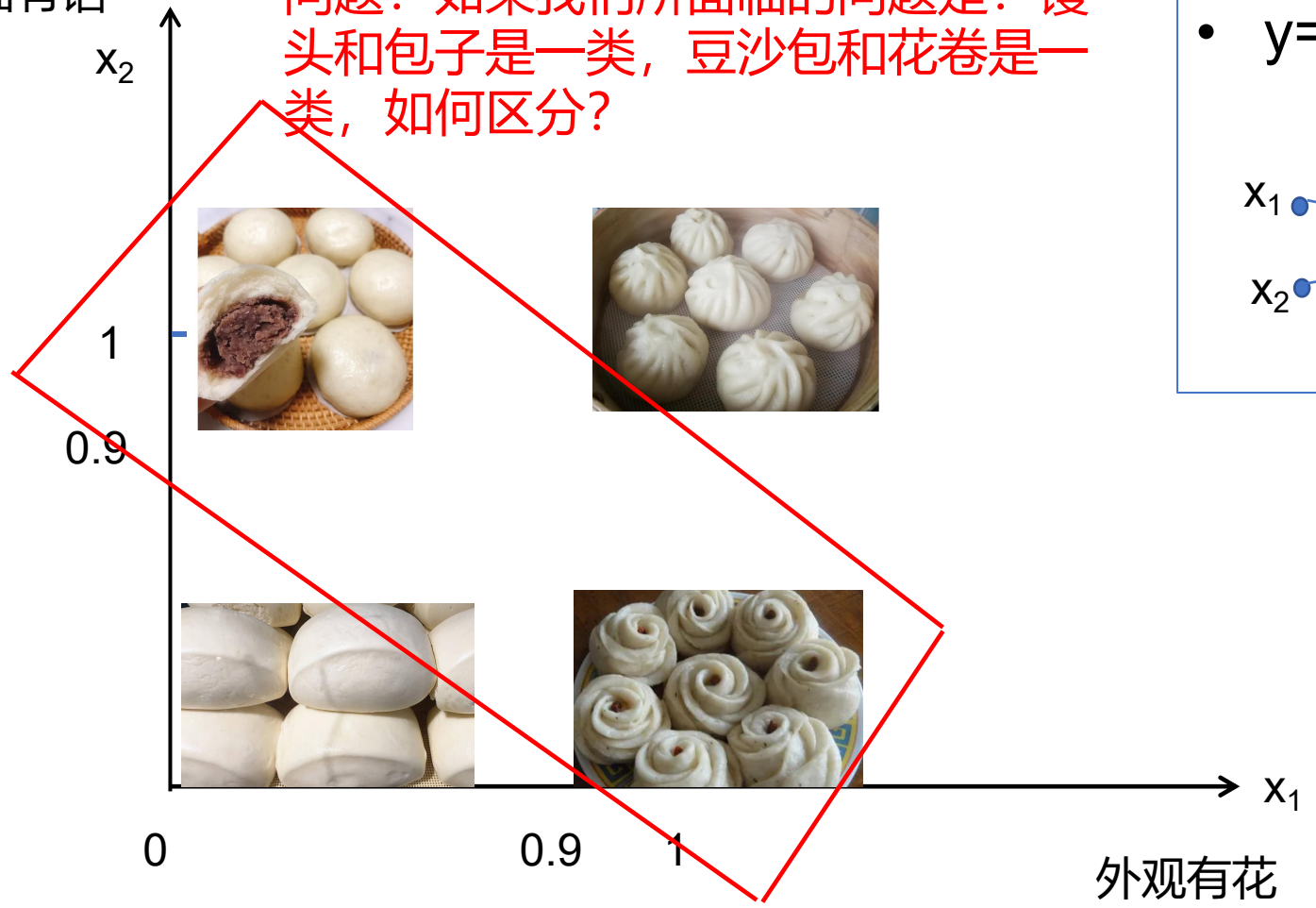
- $y=w_1 \cdot x_1+w_2 \cdot x_2+b$



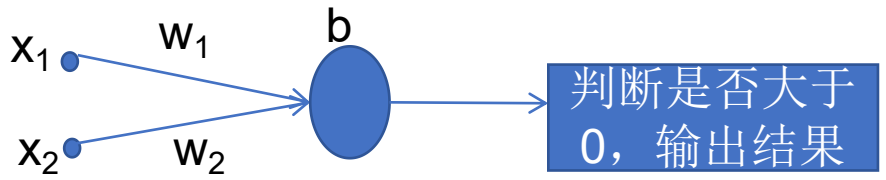
7．参数估算方法

里面有馅

问题：如果我们所面临的问题是：馒头和包子是一类，豆沙包和花卷是一类，如何区分？

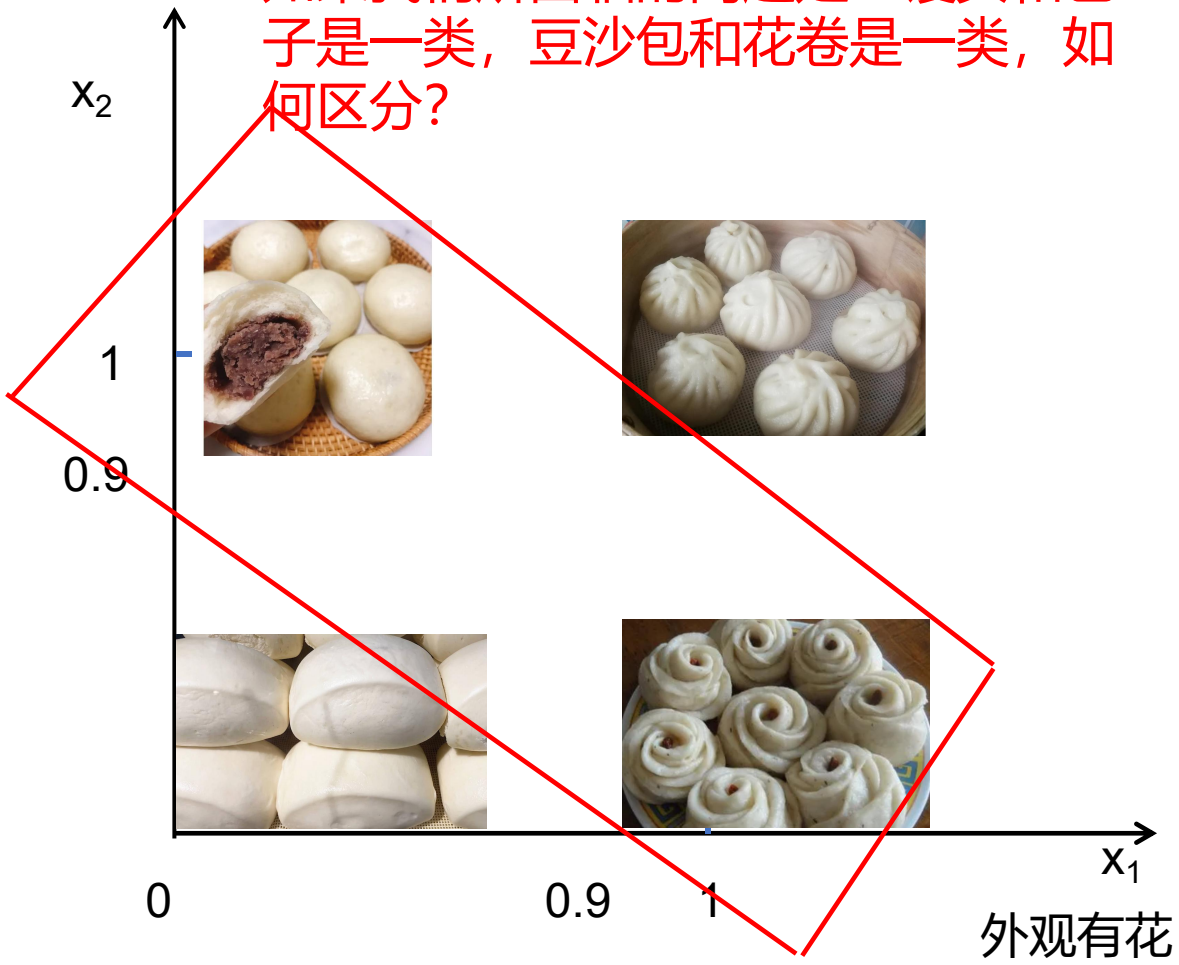


- $y=w_1 \cdot x_1+w_2 \cdot x_2+b$

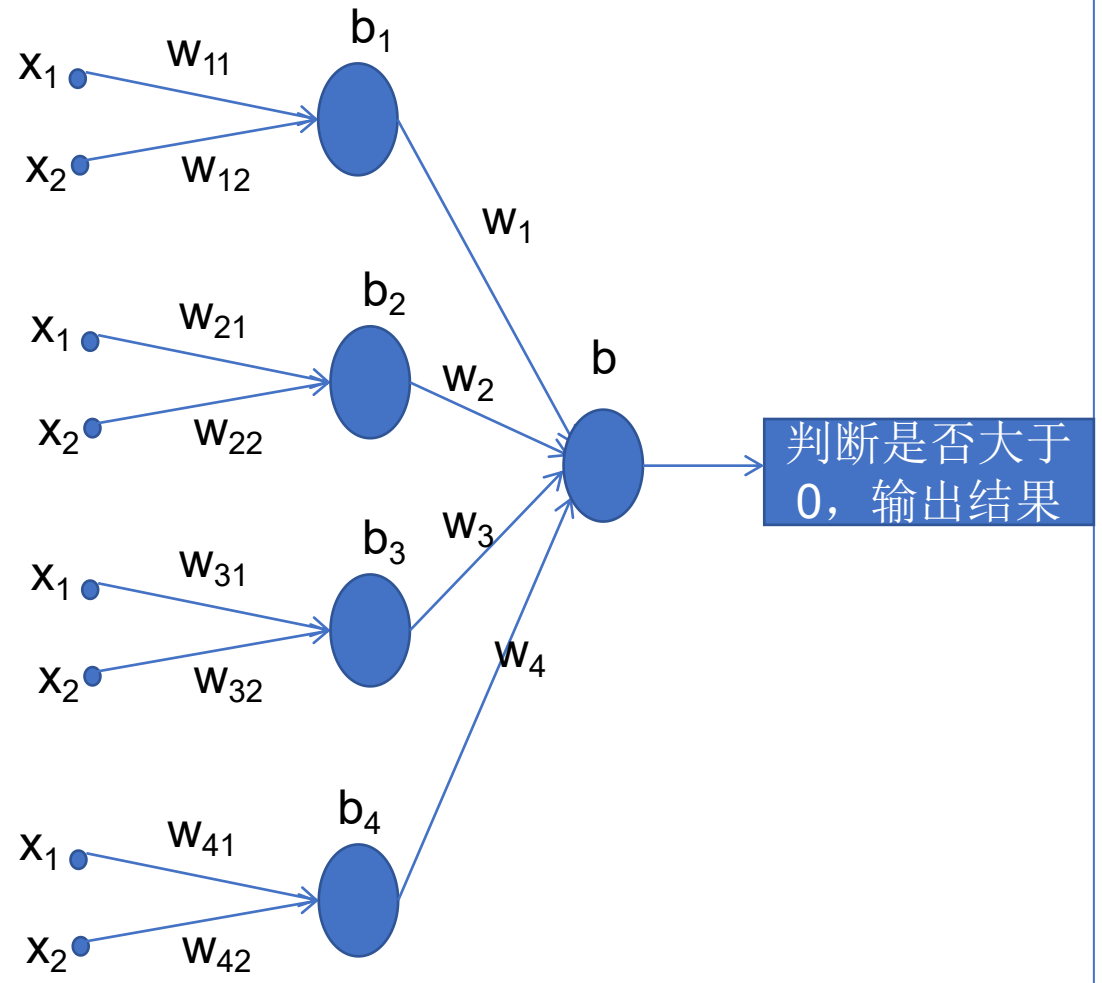


7．参数估算方法

如果我们所面临的问题是：馒头和包子是一类，豆沙包和花卷是一类，如何区分？



- y=W·X+B

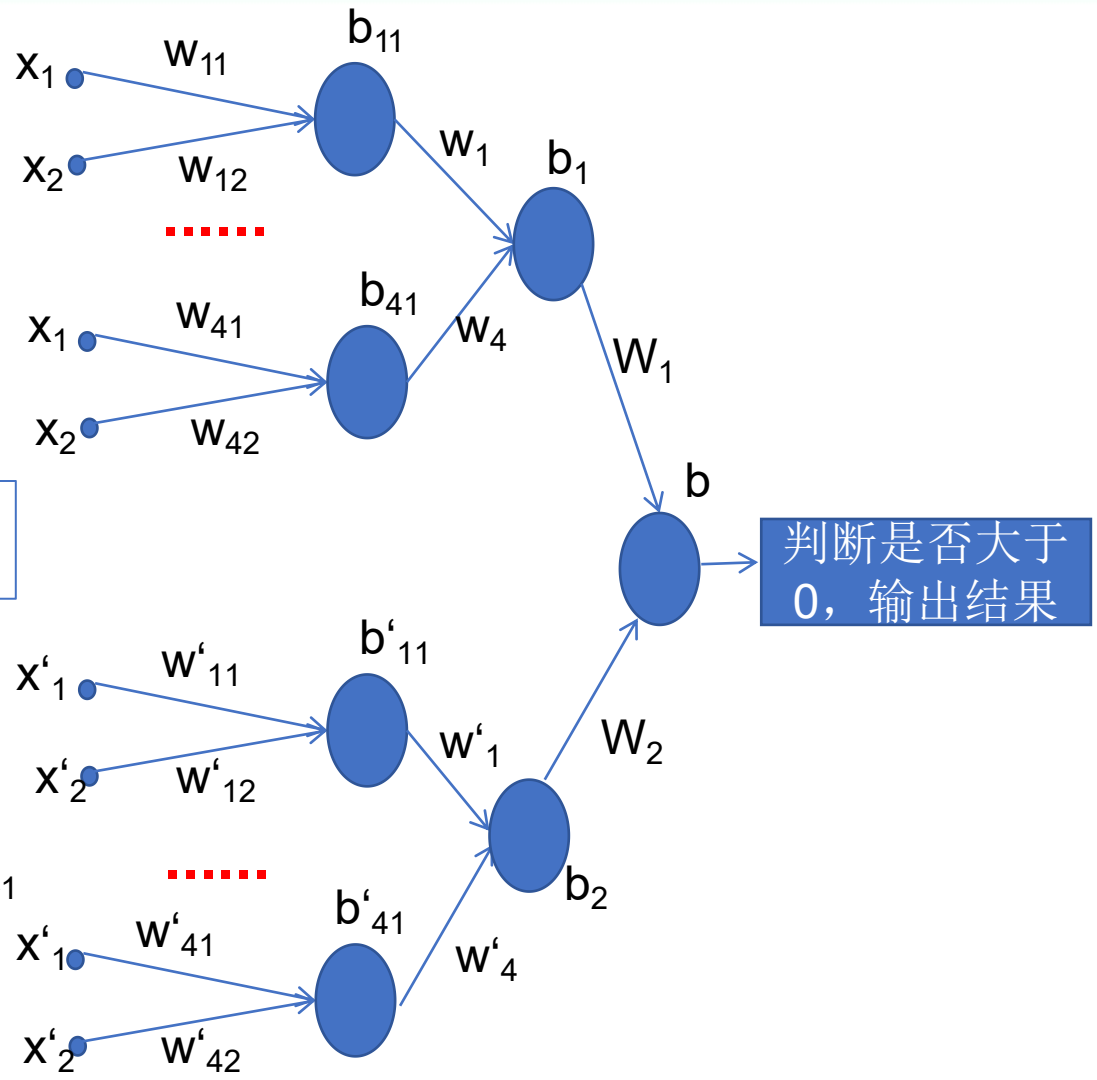


7．参数估算方法

问题：如果我们所面临的问题是：馒头和包子是一类，豆沙包和花卷是一类，如何区分？

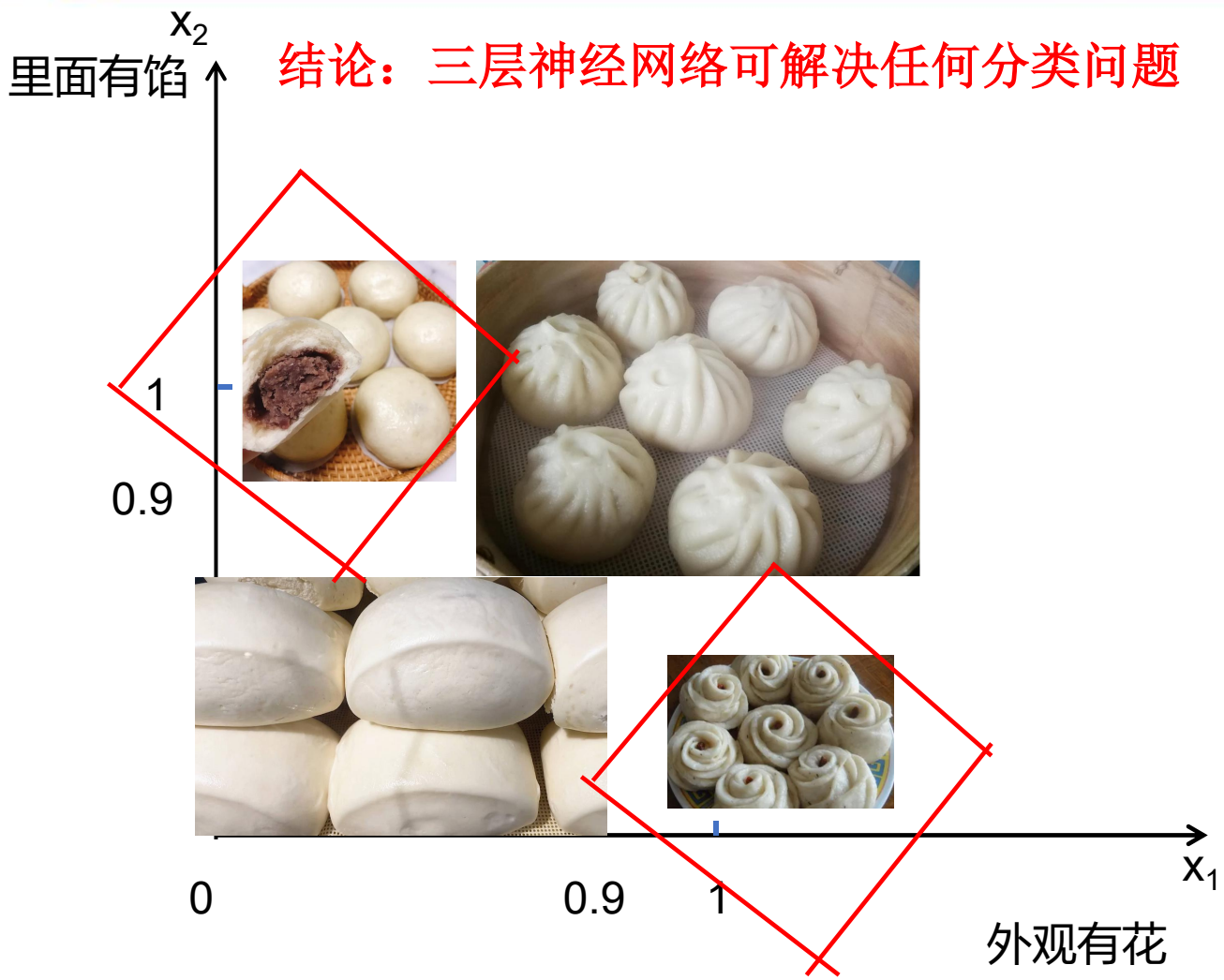


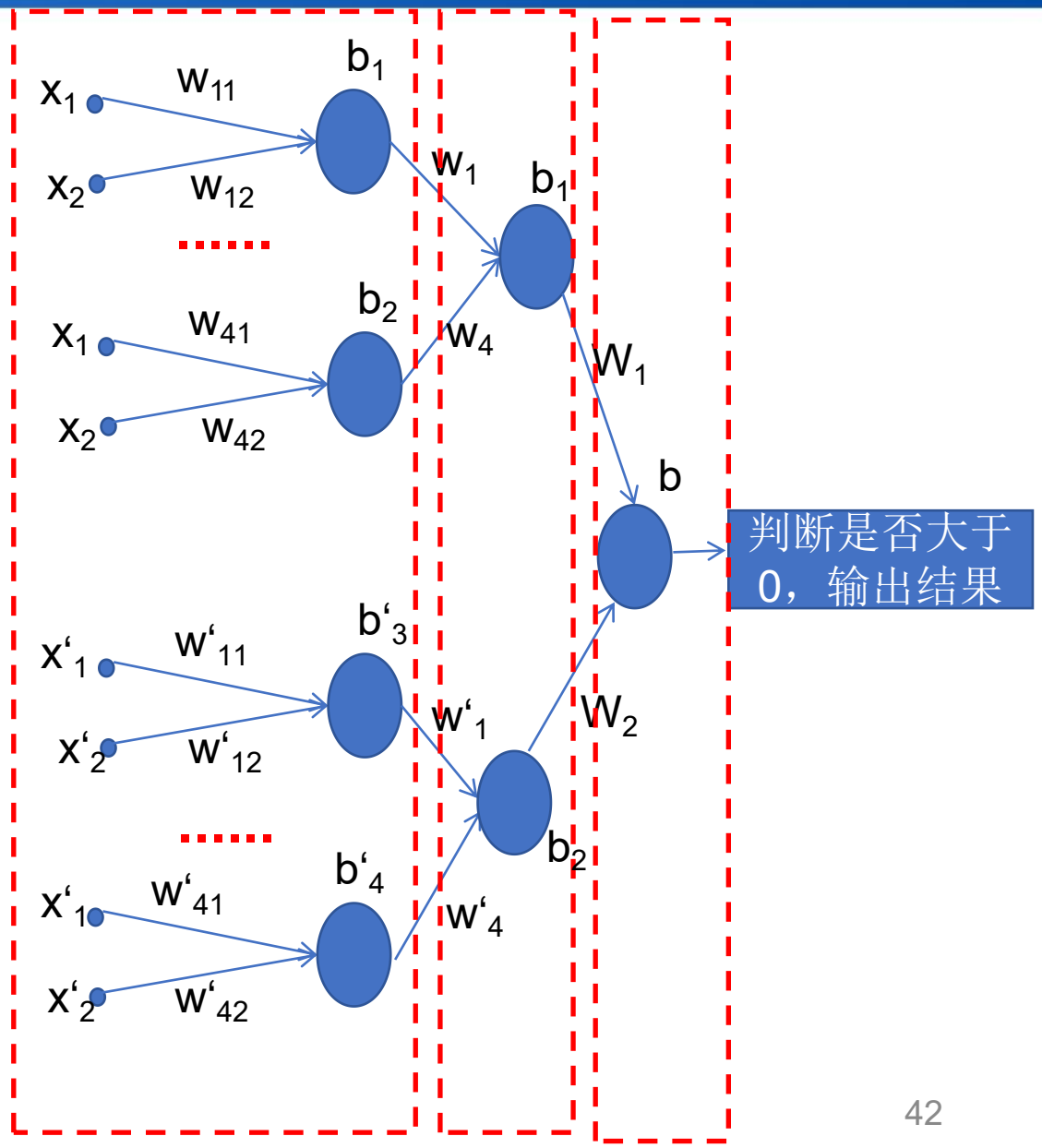
- $y=W\cdot X+B$



7．参数估算方法

结论：三层神经网络可解决任何分类问题





参数估算法

使用条件

- 具有良好的项目数据为基础
- 存在成熟的项目估算模型

特点

- 比较简单，而且也比较准确
- 如果模型选择不当或者数据不准，也会导致偏差

传 统 估 算 方 法

1. 代码行估算法
2. 功能点估算法
3. 用例点估算法
4. 类比(自顶向下)估算法
5. 自下而上估算法
6. 三点估算法
7. 参数估算法
8. 专家估算法



专家估算法

由多位专家进行成本估算，一个专家可能会有偏见，最好由多位专家进行估算，取得多个估算值，最后得出综合的估算值。

专家估算-Delphi方法



组织专家团队征求专家意见 〉 每个专家匿名、背靠背回复 〉 汇总专家意见，反馈给专家，专家修正意见，如此多轮 〉 达成共识最终决策

Delphi专家估算法-举例

某多媒体信息查询系统， 采用专家估算方法

专家1：1，8，9

（1+9+4 * 8 ）/6=7

专家2： 4， 6 ， 8

（4+8+4*6）/6=6

估算结果=（6+7）/2=6.5

传 统 估 算 方 法

1.代码行估算法：估计代码行，实现部分成本的估算

2.功能点估算法：估计功能点，实现部分成本的估算

3.用例点估算法：估计用例点，实现部分成本的估算

4.类比（自顶向下)估算法：要有经验和数据

5.自下而上估算法：较笼统

6.三点估算法：主观、笼统

7.参数估算法：相对客观，但在参数使用上避免“刻舟求剑”

8.专家估算法：主观、笼统